

Punto 2.11 - Migraciones en el modelado: ciclo completo con Soft Delete

Audiencia: Desarrolladores que ya han completado los puntos 2.1 a 2.10. Proyecto: Se utiliza el cambio real de Soft Delete para aprender el ciclo completo de una migración incremental: generar, revisar, aplicar, inspeccionar el historial, revertir, reaplicar, retirar una migración local y generar scripts SQL normales e idempotentes.

Objetivos de aprendizaje

Generar una migración incremental sobre el estado 2.10. Revisar línea a línea la migración `M2_2_11`. Comprobar el snapshot y `EFMigrationsHistory`. Aplicar y revertir el esquema hasta una migración concreta. Entender el uso seguro de `migrations remove`. Generar scripts SQL e idempotentes. Validar que el esquema resultante soporta el Soft Delete y su restauración.

Paso 1: Abrir la solución autónoma del punto

```
cd M02/PROYECTO/2.11
dotnet restore AceriaData.sln
dotnet build AceriaData.sln --configuration Release
```

`cd` → entra en el estado acumulativo 2.11. `dotnet restore` → restaura EF Core 8 y el proveedor SQL Server. `dotnet build` → comprueba que modelo, migraciones y fábrica de diseño compilan juntos.

Paso 2: Comparar el modelo 2.10 con el cambio funcional 2.11

En 2.10 existen filtros globales, pero todavía no existen las columnas persistentes del borrado lógico. El estado 2.11 añade:

```
public bool IsDeleted { get; set; }
public DateTime? DeletedAt { get; set; }
```

Estas propiedades aparecen en `OrdenFabricacion`, `PlanchaAcero`, `Aleacion` y `EstadoOrden`. Después se combinan con `HasQueryFilter` para que las consultas normales excluyan registros eliminados.

Paso 3: Generar la migración incremental al construir el estado 2.11

Al partir físicamente del estado 2.10, después de introducir las propiedades y filtros de 2.11, el comando es:

```
dotnet ef migrations add M2_2_11
```

`dotnet ef migrations add` → compara el modelo modificado con el snapshot de 2.10. `M2_2_11` → identifica el checkpoint que introduce Soft Delete. En el repositorio entregado la migración ya está generada porque 2.11 es un estado completo y reproducible. No se debe volver a ejecutar `migrations add M2_2_11` sobre el estado final: el delta original ya está registrado.

Paso 4: Revisar la migración real antes de aplicarla

Abrir `Migrations/20260927204841_M2_2_11.cs`:

```
using System;
using Microsoft.EntityFrameworkCore.Migrations;

#nullable disable

namespace AceriaData.ConsoleApp.Migrations
{
```

```
/// <inheritdoc />
public partial class M2_2_11 : Migration
{
    /// <inheritdoc />
    protected override void Up(MigrationBuilder migrationBuilder)
    {
        migrationBuilder.AddColumn<DateTime>(
            name: "DeletedAt",
            table: "PlanchasAcero",
            type: "datetime2",
            nullable: true);

        migrationBuilder.AddColumn<bool>(
            name: "IsDeleted",
            table: "PlanchasAcero",
            type: "bit",
            nullable: false,
            defaultValue: false);

        migrationBuilder.AddColumn<DateTime>(
            name: "DeletedAt",
            table: "OrdenesFabricacion",
            type: "datetime2",
            nullable: true);

        migrationBuilder.AddColumn<bool>(
            name: "IsDeleted",
            table: "OrdenesFabricacion",
            type: "bit",
            nullable: false,
            defaultValue: false);

        migrationBuilder.AddColumn<DateTime>(
            name: "DeletedAt",
            table: "EstadosOrden",
            type: "datetime2",
            nullable: true);

        migrationBuilder.AddColumn<bool>(
            name: "IsDeleted",
            table: "EstadosOrden",
            type: "bit",
            nullable: false,
            defaultValue: false);

        migrationBuilder.AddColumn<DateTime>(
            name: "DeletedAt",
            table: "Aleaciones",
            type: "datetime2",
            nullable: true);

        migrationBuilder.AddColumn<bool>(
            name: "IsDeleted",
            table: "Aleaciones",
            type: "bit",
            nullable: false,
            defaultValue: false);
    }

    /// <inheritdoc />
    protected override void Down(MigrationBuilder migrationBuilder)
    {
        migrationBuilder.DropColumn(
            name: "DeletedAt",
            table: "PlanchasAcero");

        migrationBuilder.DropColumn(
            name: "IsDeleted",
            table: "PlanchasAcero");

        migrationBuilder.DropColumn(
            name: "DeletedAt",
            table: "OrdenesFabricacion");
    }
}
```

```

        migrationBuilder.DropColumn(
            name: "IsDeleted",
            table: "OrdenesFabricacion");

        migrationBuilder.DropColumn(
            name: "DeletedAt",
            table: "EstadosOrden");

        migrationBuilder.DropColumn(
            name: "IsDeleted",
            table: "EstadosOrden");

        migrationBuilder.DropColumn(
            name: "DeletedAt",
            table: "Aleaciones");

        migrationBuilder.DropColumn(
            name: "IsDeleted",
            table: "Aleaciones");
    }
}

```

Explicación línea a línea de la migración M2_2_11

Línea 1: `using System;` → importa los tipos base de .NET necesarios para las columnas DateTime.

Línea 2: `using Microsoft.EntityFrameworkCore.Migrations;` → importa `Migration` y `MigrationBuilder`, la API que expresa cambios de esquema.

Línea 4: `#nullable disable` → mantiene el contexto de nulabilidad generado por EF Core para este archivo.

Línea 6: `namespace AceriaData.ConsoleApp.Migrations` → declara el espacio de nombres del historial de migraciones del contexto.

Línea 7: `{` → abre el bloque de la declaración o método anterior.

Línea 8: `/// <inheritdoc />` → comentario XML generado automáticamente para documentar el miembro siguiente.

Línea 9: `public partial class M2_2_11 : Migration` → declara la migración incremental correspondiente al checkpoint 2.11.

Línea 10: `{` → abre el bloque de la declaración o método anterior.

Línea 11: `/// <inheritdoc />` → comentario XML generado automáticamente para documentar el miembro siguiente.

Línea 12: `protected override void Up(MigrationBuilder migrationBuilder)` → define las operaciones que llevan el esquema desde 2.10 hasta 2.11.

Línea 13: `{` → abre el bloque de la declaración o método anterior.

Línea 14: `migrationBuilder.AddColumn<DateTime>(` → inicia la adición de una columna `DeletedAt` de fecha/hora.

Línea 15: `name: "DeletedAt",` → identifica la columna afectada como `DeletedAt`.

Línea 16: `table: "PlanchasAcero",` → dirige la operación a la tabla `PlanchasAcero`.

Línea 17: `type: "datetime2",` → fija el tipo SQL Server `datetime2`.

Línea 18: `nullable: true);` → `DeletedAt` admite NULL mientras el registro no esté eliminado.

Línea 20: `migrationBuilder.AddColumn<bool>(` → inicia la adición de una columna `IsDeleted` booleana.

Línea 21: `name: "IsDeleted",` → identifica la columna afectada como `IsDeleted`.

Línea 22: `table: "PlanchasAcero",` → dirige la operación a la tabla `PlanchasAcero`.

Línea 23: `type: "bit",` → fija el tipo SQL Server `bit`.

Línea 24: `nullable: false,` → `IsDeleted` es obligatorio para todas las filas.

Línea 25: `defaultValue: false);` → rellena las filas existentes con `false` para mantenerlas activas al aplicar el cambio.

Línea 27: `migrationBuilder.AddColumn<DateTime>(` → inicia la adición de una columna `DeletedAt` de fecha/hora.

Línea 28: `name: "DeletedAt",` → identifica la columna afectada como `DeletedAt`.

Línea 29: `table: "OrdenesFabricacion",` → dirige la operación a la tabla `OrdenesFabricacion`.

Línea 30: `type: "datetime2",` → fija el tipo SQL Server `datetime2`.

Línea 31: `nullable: true);` → `DeletedAt` admite NULL mientras el registro no esté eliminado.

Línea 33: `migrationBuilder.AddColumn<bool>(` → inicia la adición de una columna `IsDeleted` booleana.

Línea 34: `name: "IsDeleted",` → identifica la columna afectada como `IsDeleted`.

Línea 35: `table: "OrdenesFabricacion",` → dirige la operación a la tabla `OrdenesFabricacion`.

Línea 36: `type: "bit",` → fija el tipo SQL Server `bit`.

Línea 37: `nullable: false,` → `IsDeleted` es obligatorio para todas las filas.

Línea 38: `defaultValue: false);` → rellena las filas existentes con `false` para mantenerlas activas al aplicar el cambio.

Línea 40: `migrationBuilder.AddColumn<DateTime>(` → inicia la adición de una columna `DeletedAt` de fecha/hora.

Línea 41: `name: "DeletedAt",` → identifica la columna afectada como `DeletedAt`.

Línea 42: `table: "EstadosOrden",` → dirige la operación a la tabla `EstadosOrden`.

Línea 43: `type: "datetime2",` → fija el tipo SQL Server `datetime2`.

Línea 44: `nullable: true);` → `DeletedAt` admite NULL mientras el registro no esté eliminado.

Línea 46: `migrationBuilder.AddColumn<bool>(` → inicia la adición de una columna `IsDeleted` booleana.

Línea 47: `name: "IsDeleted",` → identifica la columna afectada como `IsDeleted`.

Línea 48: `table: "EstadosOrden",` → dirige la operación a la tabla `EstadosOrden`.

Línea 49: `type: "bit",` → fija el tipo SQL Server `bit`.

Línea 50: `nullable: false,` → `IsDeleted` es obligatorio para todas las filas.

Línea 51: `defaultValue: false);` → rellena las filas existentes con `false` para mantenerlas activas al aplicar el cambio.

Línea 53: `migrationBuilder.AddColumn<DateTime>(` → inicia la adición de una columna `DeletedAt` de fecha/hora.

Línea 54: `name: "DeletedAt",` → identifica la columna afectada como `DeletedAt`.

Línea 55: `table: "Aleaciones",` → dirige la operación a la tabla `Aleaciones`.

Línea 56: `type: "datetime2",` → fija el tipo SQL Server `datetime2`.

Línea 57: `nullable: true);` → `DeletedAt` admite NULL mientras el registro no esté eliminado.

Línea 59: `migrationBuilder.AddColumn<bool>(` → inicia la adición de una columna `IsDeleted` booleana.

Línea 60: `name: "IsDeleted",` → identifica la columna afectada como `IsDeleted`.

Línea 61: `table: "Aleaciones"`, → dirige la operación a la tabla `Aleaciones`.

Línea 62: `type: "bit"`, → fija el tipo SQL Server `bit`.

Línea 63: `nullable: false`, → `IsDeleted` es obligatorio para todas las filas.

Línea 64: `defaultValue: false`); → rellena las filas existentes con `false` para mantenerlas activas al aplicar el cambio.

Línea 65: `}` → cierra el bloque de la declaración o método anterior.

Línea 67: `/// <inheritdoc />` → comentario XML generado automáticamente para documentar el miembro siguiente.

Línea 68: `protected override void Down(MigrationBuilder migrationBuilder)` → define las operaciones inversas que devuelven el esquema desde 2.11 hasta 2.10.

Línea 69: `{` → abre el bloque de la declaración o método anterior.

Línea 70: `migrationBuilder.DropColumn()` → inicia la eliminación de una de las columnas añadidas por esta migración.

Línea 71: `name: "DeletedAt"`, → identifica la columna afectada como `DeletedAt`.

Línea 72: `table: "PlanchasAcero"`); → dirige la operación a la tabla `PlanchasAcero`.

Línea 74: `migrationBuilder.DropColumn()` → inicia la eliminación de una de las columnas añadidas por esta migración.

Línea 75: `name: "IsDeleted"`, → identifica la columna afectada como `IsDeleted`.

Línea 76: `table: "PlanchasAcero"`); → dirige la operación a la tabla `PlanchasAcero`.

Línea 78: `migrationBuilder.DropColumn()` → inicia la eliminación de una de las columnas añadidas por esta migración.

Línea 79: `name: "DeletedAt"`, → identifica la columna afectada como `DeletedAt`.

Línea 80: `table: "OrdenesFabricacion"`); → dirige la operación a la tabla `OrdenesFabricacion`.

Línea 82: `migrationBuilder.DropColumn()` → inicia la eliminación de una de las columnas añadidas por esta migración.

Línea 83: `name: "IsDeleted"`, → identifica la columna afectada como `IsDeleted`.

Línea 84: `table: "OrdenesFabricacion"`); → dirige la operación a la tabla `OrdenesFabricacion`.

Línea 86: `migrationBuilder.DropColumn()` → inicia la eliminación de una de las columnas añadidas por esta migración.

Línea 87: `name: "DeletedAt"`, → identifica la columna afectada como `DeletedAt`.

Línea 88: `table: "EstadosOrden"`); → dirige la operación a la tabla `EstadosOrden`.

Línea 90: `migrationBuilder.DropColumn()` → inicia la eliminación de una de las columnas añadidas por esta migración.

Línea 91: `name: "IsDeleted"`, → identifica la columna afectada como `IsDeleted`.

Línea 92: `table: "EstadosOrden"`); → dirige la operación a la tabla `EstadosOrden`.

Línea 94: `migrationBuilder.DropColumn()` → inicia la eliminación de una de las columnas añadidas por esta migración.

Línea 95: `name: "DeletedAt"`, → identifica la columna afectada como `DeletedAt`.

Línea 96: `table: "Aleaciones"`); → dirige la operación a la tabla `Aleaciones`.

Línea 98: `migrationBuilder.DropColumn()` → inicia la eliminación de una de las columnas añadidas por esta migración.

Línea 99: `name: "IsDeleted"`, → identifica la columna afectada como `IsDeleted`.

Línea 100: `table: "Aleaciones"`; → dirige la operación a la tabla `Aleaciones`.

Línea 101: `}` → cierra el bloque de la declaración o método anterior.

Línea 102: `}` → cierra el bloque de la declaración o método anterior.

Línea 103: `}` → cierra el bloque de la declaración o método anterior.

Las columnas `IsDeleted` usan `defaultValue: false`. Así, las filas existentes permanecen activas después del cambio de esquema.

Paso 5: Aplicar la migración

```
dotnet ef database update --configuration Release
```

Resultado esperado: EF Core aplica las migraciones pendientes hasta `M2_2_11`. Las cuatro tablas afectadas contienen `IsDeleted` y `DeletedAt`.

Paso 6: Verificar la cadena y la tabla de historial

```
dotnet ef migrations list --configuration Release
```

Consultar además en SQL Server LocalDB:

```
SELECT MigrationId, ProductVersion
FROM __EFMigrationsHistory
ORDER BY MigrationId;
```

Resultado esperado: `M2_2_11` aparece como última migración aplicada.

Paso 7: Revertir exactamente hasta 2.10

```
dotnet ef database update 20260927204833_M2_2_10 --configuration Release
```

Resultado esperado: EF Core ejecuta el `Down` de `M2_2_11`, elimina las columnas añadidas en este punto y retira la migración de `__EFMigrationsHistory`.

Paso 8: Reaplicar 2.11

```
dotnet ef database update --configuration Release
```

Resultado esperado: `Up` vuelve a crear las columnas y el historial vuelve a registrar `M2_2_11`.

Paso 9: Comprender migrations remove sin dañar el repositorio canónico

`migrations remove` modifica archivos fuente y snapshot; por eso no se ejecuta directamente sobre la copia canónica ya validada. Para practicarlo, trabajar sobre una copia desechable de `2.11`, revertir primero la base de laboratorio a `2.10` y ejecutar:

```
dotnet ef database update 20260927204833_M2_2_10 --configuration Release
dotnet ef migrations remove --configuration Release
```

Resultado esperado en la copia: desaparecen los archivos de la última migración y el snapshot vuelve al modelo anterior. Después se descarta la copia. Si una migración ya se ha compartido o desplegado, se crea una migración correctiva nueva en lugar de reescribir la historia.

Paso 10: Revisar el ModelSnapshot

Localizar en `Migrations/AceriaDbContextModelSnapshot.cs` las propiedades nuevas. Un fragmento real del snapshot es:

```
b.Property<DateTime?>("DeletedAt")
    .HasColumnType("datetime2");

b.Property<string>("Descripcion")
    .HasMaxLength(500)
    .HasColumnType("nvarchar(500)");

b.Property<bool>("IsDeleted")
    .HasColumnType("bit");

b.Property<string>("Nombre")
    .IsRequired()
    .HasMaxLength(100)
    .HasColumnType("nvarchar(100)");

b.Property<double>("PorcentajeCarbono")
    .HasColumnType("float");

b.Property<double>("PorcentajeManganeso")
    .HasColumnType("float");

b.HasKey("Id");
```

El snapshot no es la base ni el historial aplicado: es la referencia de modelo que la siguiente ejecución de `migrations add` comparará con el nuevo modelo.

Paso 11: Generar scripts SQL normal e idempotente

```
dotnet ef migrations script 20260927204833_M2_2_10 20260927204841_M2_2_11 --configuration Release --
output softdelete.sql
dotnet ef migrations script --idempotent --configuration Release --output migraciones_idempotentes.sql
```

`softdelete.sql` → contiene sólo el tramo 2.10 → 2.11. `migraciones_idempotentes.sql` → comprueba `__EFMigrationsHistory` antes de ejecutar cada migración.

Paso 12: Ejecutar la comprobación funcional del Soft Delete

```
dotnet run --project AceriaData.Console.csproj --configuration Release --no-build
```

Resultado esperado:

```
2.11 OK | Tras borrar visibles: 0 | Totales: 1 | Restaurada: True
```

La ejecución demuestra que el cambio de esquema es funcional: el filtro oculta la entidad marcada, `IgnoreQueryFilters()` permite recuperarla y la restauración vuelve a hacerla visible.

Errores comunes

Error	Causa	Solución
Migración vacía	El modelo y el snapshot ya representan el mismo estado	Generarla durante la transición real 2.10 → 2.11

Error	Causa	Solución
Columna no anulable incompatible con datos existentes	No existe valor válido para las filas previas	Diseñar default, backfill o transición nullable
<code>database update</code> no cambia nada	La migración ya figura en <code>__EFMigrationsHistory</code>	Comprobar el historial y la base objetivo
La reversión pierde información	<code>Down</code> elimina columnas o tablas	Revisar impacto de datos y disponer de backup
<code>migrations remove</code> no es apropiado	La migración ya se compartió o desplegó	Crear una migración correctiva en vez de reescribir la historia
Script idempotente incompleto	Se está usando otro contexto/proyecto	Verificar proyecto, contexto y cadena

Retorno resuelto

Generar únicamente el SQL del delta 2.10 → 2.11 y localizar en el script las cuatro parejas `DeletedAt / IsDeleted`. Después comprobar que el script idempotente consulta `__EFMigrationsHistory` antes de ejecutar los bloques. La solución es usar el rango explícito para el primer script y `--idempotent` para el segundo.

Analogía final

El modelo es el plano actual de una línea de producción; el snapshot es el plano archivado después de la última reforma; `M2_2_11` es la orden de obra que transforma 2.10 en 2.11; y `__EFMigrationsHistory` es el libro de reformas realmente ejecutadas. Soft Delete es la reforma concreta con la que se estudia todo el ciclo.

Resultado esperado

Al terminar el punto se sabe generar y revisar una migración incremental, comprobar `Up` y `Down`, leer el snapshot, aplicar y revertir hasta un checkpoint, entender `migrations remove`, producir SQL e idempotentes y validar funcionalmente el Soft Delete.

Conexión con el siguiente punto

2.12 reorganiza el mismo sistema en Domain, Application, Infrastructure y Console. Las migraciones pasan a Infrastructure y el contexto de diseño se resuelve mediante `IDesignTimeDbContextFactory`, pero el historial y el esquema continúan siendo los validados en 2.11.

Código acumulativo completo del estado 2.11

El siguiente archivo es el `Program.cs` real del estado ejecutable del punto. Se incluye para que la práctica y el proyecto ejecutable no diverjan.

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;

namespace AceriaData.ConsoleApp;

[Table("OrdenesFabricacion")]
public class OrdenFabricacion
{
    [Key]
    public int Id { get; set; }
    [Required]
```

```

    [MaxLength(50)]
    public string NumeroOrden { get; set; } = string.Empty;
    [Required]
    [MaxLength(200)]
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
    [Required]
    [MaxLength(50)]
    public string Estado { get; set; } = "Pendiente";
    [MaxLength(500)]
    public string? Observaciones { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
    public DetalleOrden? Detalle { get; set; }
    public CertificadoCalidad? Certificado { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
    public bool IsDeleted { get; set; }
    public DateTime? DeletedAt { get; set; }
}

[Table("PlanchasAcero")]
public class PlanchaAcero
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    [Precision(18, 3)]
    public decimal Peso { get; set; }
    public bool Activa { get; set; } = true;
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
    public bool IsDeleted { get; set; }
    public DateTime? DeletedAt { get; set; }
}

[Table("Aleaciones")]
public class Aleacion
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(100)]
    public string Nombre { get; set; } = string.Empty;
    [Required]
    [MaxLength(20)]
    public string Codigo { get; set; } = string.Empty;
    public double PorcentajeCarbono { get; set; }
    public double PorcentajeManganeso { get; set; }
    [MaxLength(500)]
    public string? Descripcion { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
    public bool IsDeleted { get; set; }
    public DateTime? DeletedAt { get; set; }
}

[Table("EstadosOrden")]
public class EstadoOrden
{
    [Key]
    public int Id { get; set; }
    [Required]
    [MaxLength(50)]
    public string Nombre { get; set; } = string.Empty;
    [MaxLength(250)]
    public string Descripcion { get; set; } = string.Empty;
    public bool Activo { get; set; } = true;
    public bool IsDeleted { get; set; }
    public DateTime? DeletedAt { get; set; }
}

```

```

[Table("DetallesOrden")]
public class DetalleOrden
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    [Required]
    [MaxLength(200)]
    public string ComposicionQuimica { get; set; } = string.Empty;
    public double TemperaturaColada { get; set; }
    [MaxLength(500)]
    public string? Notas { get; set; }
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[Table("CertificadosCalidad")]
public class CertificadoCalidad
{
    [Key]
    public int Id { get; set; }
    public int OrdenId { get; set; }
    [Required]
    [MaxLength(50)]
    public string NumeroCertificado { get; set; } = string.Empty;
    public DateTime FechaEmision { get; set; }
    [Required]
    [MaxLength(100)]
    public string OrganismoCertificador { get; set; } = string.Empty;
    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}

[PrimaryKey(nameof(OrdenFabricacionId), nameof(AleacionId))]
[Table("OrdenesAleaciones")]
public class OrdenAleacion
{
    public int OrdenFabricacionId { get; set; }
    public int AleacionId { get; set; }
    public DateTime FechaAsignacion { get; set; } = DateTime.Now;
    [Precision(18, 3)]
    public decimal CantidadUtilizada { get; set; }
    [MaxLength(20)]
    public string EstadoRelacion { get; set; } = "Activa";
    public OrdenFabricacion Orden { get; set; } = null!;
    public Aleacion Aleacion { get; set; } = null!;
}

public class AceriaDbContext : DbContext
{
    public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }

    public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();
    public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();
    public DbSet<Aleacion> Aleaciones => Set<Aleacion>();
    public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();
    public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();
    public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();
    public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        modelBuilder.Entity<OrdenFabricacion>(entity =>
        {
            entity.ToTable("OrdenesFabricacion");
            entity.HasKey(o => o.Id);
            entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);
            entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);
            entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");
            entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");
            entity.Property(o => o.Observaciones).HasMaxLength(500);
        });
    }
}

```

```

        entity.HasAlternateKey(o => o.NumeroOrden).HasName("AK_OrdenesFabricacion_NumeroOrden");
        entity.HasIndex(o => o.Cliente).HasDatabaseName("IX_OrdenesFabricacion_Cliente");
        entity.HasIndex(o => new { o.Cliente,
o.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Cliente_FechaCreacion");
        entity.HasIndex(o => o.FechaEntrega).HasFilter("[Estado] =
'Pendiente'").HasDatabaseName("IX_OrdenesFabricacion_FechaEntrega_Pendientes");
        entity.HasIndex(o => o.Estado).IncludeProperties(o => new { o.NumeroOrden, o.Cliente,
o.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Estado_Incluye");
        entity.HasQueryFilter(o => !o.IsDeleted && o.Estado != "Cancelada");
    });
    modelBuilder.Entity<PlanchaAcero>(entity =>
    {
        entity.ToTable("PlanchasAcero", t =>
        {
            t.HasCheckConstraint("CK_PlanchasAcero_Espesor", "[Espesor] > 0");
            t.HasCheckConstraint("CK_PlanchasAcero_Ancho", "[Ancho] > 0");
            t.HasCheckConstraint("CK_PlanchasAcero_Largo", "[Largo] > 0");
            t.HasCheckConstraint("CK_PlanchasAcero_Peso", "[Peso] > 0");
        });
        entity.HasKey(x => x.Id);
        entity.Property(x => x.Peso).HasPrecision(18, 3);
        entity.Property(x => x.Activa).HasDefaultValue(true);
        entity.HasOne(x => x.Orden)
            .WithMany(o => o.Planchas)
            .HasForeignKey(x => x.OrdenId)
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();
        entity.HasIndex(x => new { x.OrdenId,
x.Activa }).HasDatabaseName("IX_PlanchasAcero_OrdenId_Activa");
        entity.HasIndex(x => x.Espesor).HasFilter("[Activa] =
1").HasDatabaseName("IX_PlanchasAcero_Espesor_Activas");
        entity.HasQueryFilter(x => !x.IsDeleted && x.Activa);
    });
    modelBuilder.Entity<Aleacion>(entity =>
    {
        entity.ToTable("Aleaciones", t =>
        {
            t.HasCheckConstraint("CK_Aleaciones_PorcentajeCarbono", "[PorcentajeCarbono] >= 0 AND
[PorcentajeCarbono] <= 2");
            t.HasCheckConstraint("CK_Aleaciones_PorcentajeManganeso", "[PorcentajeManganeso] >= 0
AND [PorcentajeManganeso] <= 5");
        });
        entity.HasKey(a => a.Id);
        entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);
        entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);
        entity.Property(a => a.Descripcion).HasMaxLength(500);
        entity.HasAlternateKey(a => a.Codigo).HasName("AK_Aleaciones_Codigo");
        entity.HasIndex(a => a.Nombre).HasDatabaseName("IX_Aleaciones_Nombre");
        entity.HasIndex(a => new { a.PorcentajeCarbono,
a.PorcentajeManganeso }).HasDatabaseName("IX_Aleaciones_Porcentajes");
        entity.HasQueryFilter(a => !a.IsDeleted);
    });
    modelBuilder.Entity<EstadoOrden>(entity =>
    {
        entity.ToTable("EstadosOrden");
        entity.HasKey(e => e.Id);
        entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);
        entity.Property(e => e.Descripcion).HasMaxLength(250);
        entity.Property(e => e.Activo).HasDefaultValue(true);
        entity.HasQueryFilter(e => !e.IsDeleted && e.Activo);
    });
    modelBuilder.Entity<DetalleOrden>(entity =>
    {
        entity.ToTable("DetallesOrden");
        entity.HasKey(d => d.Id);
        entity.Property(d => d.ComposicionQuimica).IsRequired().HasMaxLength(200);
        entity.Property(d => d.Notas).HasMaxLength(500);
        entity.HasOne(d => d.Orden)
            .WithOne(o => o.Detalle)
            .HasForeignKey<DetalleOrden>(d => d.OrdenId)
            .OnDelete(DeleteBehavior.Cascade)
            .IsRequired();
        entity.HasQueryFilter(d => !d.Orden.IsDeleted && d.Orden.Estado != "Cancelada");
    });

```

```

});

modelBuilder.Entity<CertificadoCalidad>(entity =>
{
    entity.ToTable("CertificadosCalidad");
    entity.HasKey(c => c.Id);
    entity.Property(c => c.NumeroCertificado).IsRequired().HasMaxLength(50);
    entity.Property(c => c.OrganismoCertificador).IsRequired().HasMaxLength(100);
    entity.HasAlternateKey(c =>
c.NumeroCertificado).HasName("AK_CertificadosCalidad_NumeroCertificado");
    entity.HasIndex(c => c.FechaEmision).HasDatabaseName("IX_CertificadosCalidad_FechaEmision");
    entity.HasOne(c => c.Orden)
        .WithOne(o => o.Certificado)
        .HasForeignKey<CertificadoCalidad>(c => c.OrdenId)
        .OnDelete(DeleteBehavior.Cascade)
        .IsRequired();
    entity.HasQueryFilter(c => !c.Orden.IsDeleted && c.Orden.Estado != "Cancelada");
});
modelBuilder.Entity<OrdenAleacion>(entity =>
{
    entity.ToTable("OrdenesAleaciones");
    entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });
    entity.Property(x => x.FechaAsignacion).HasDefaultValueSql("GETDATE()");
    entity.Property(x => x.CantidadUtilizada).HasPrecision(18, 3);
    entity.Property(x =>
x.EstadoRelacion).IsRequired().HasMaxLength(20).HasDefaultValue("Activa");
    entity.HasOne(x => x.Orden)
        .WithMany(o => o.OrdenesAleaciones)
        .HasForeignKey(x => x.OrdenFabricacionId)
        .OnDelete(DeleteBehavior.Cascade);
    entity.HasOne(x => x.Aleacion)
        .WithMany(a => a.OrdenesAleaciones)
        .HasForeignKey(x => x.AleacionId)
        .OnDelete(DeleteBehavior.Restrict);
    entity.HasIndex(x => x.AleacionId).HasDatabaseName("IX_OrdenesAleaciones_AleacionId");
    entity.HasIndex(x => x.EstadoRelacion).HasFilter("[EstadoRelacion] =
'Activa'").HasDatabaseName("IX_OrdenesAleaciones_EstadoRelacion_Activas");
    entity.HasQueryFilter(x => x.EstadoRelacion == "Activa" && !x.Orden.IsDeleted && !
x.Aleacion.IsDeleted);
});
}
}

public interface IOordenRepositorio
{
    List<OrdenFabricacion> ObtenerTodas();
    OrdenFabricacion? ObtenerPorId(int id);
    OrdenFabricacion? ObtenerPorNumero(string numeroOrden);
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
    void EliminarLogicamente(int id);
    void Restaurar(int id);

    int Guardar();
}

public sealed class OrdenRepositorio : IOordenRepositorio
{
    private readonly AceriaDbContext _context;
    public OrdenRepositorio(AceriaDbContext context) => _context = context;

    public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o =>
o.Id).ToList();
    public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);
    public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) =>
_context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);
    public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);
    public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);
    public void EliminarLogicamente(int id)
    {
        var orden = _context.OrdenesFabricacion.FirstOrDefault(o => o.Id == id);
        if (orden is null) return;
        orden.IsDeleted = true;
    }
}

```

```

        orden.DeletedAt = DateTime.UtcNow;
    }

    public void Restaurar(int id)
    {
        var orden = _context.OrdenesFabricacion.IgnoreQueryFilters().FirstOrDefault(o => o.Id == id);
        if (orden is null) return;
        orden.IsDeleted = false;
        orden.DeletedAt = null;
    }

    public int Guardar() => _context.SaveChanges();
}

public interface IServicioOrdenes
{
    string ObtenerResumen(int ordenId);
}

public sealed class ServicioOrdenes : IServicioOrdenes
{
    private readonly IOOrdenRepositorio _repositorio;
    private readonly AceriaDbContext _context;
    public ServicioOrdenes(IOOrdenRepositorio repositorio, AceriaDbContext context)
    {
        _repositorio = repositorio;
        _context = context;
    }

    public string ObtenerResumen(int ordenId)
    {
        var orden = _repositorio.ObtenerPorId(ordenId);
        if (orden is null) return $"Orden {ordenId} no encontrada";
        var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);
        return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";
    }
}

public static class Program
{
    public static void Main()
    {
        var configuration = new ConfigurationBuilder()
            .SetBasePath(AppContext.BaseDirectory)
            .AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)
            .AddEnvironmentVariables()
            .Build();

        var connectionString = configuration.GetConnectionString("AceriaDB")
            ?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");

        var services = new ServiceCollection();
        services.AddDbContext<AceriaDbContext>(options =>
            options
                .UseSqlServer(connectionString, sql =>
                    {
                        sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);
                        sql.CommandTimeout(60);
                    })
                .LogTo(global::System.Console.WriteLine, new[] {
                    "Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information)
                .EnableDetailedErrors());

        services.AddScoped<IOOrdenRepositorio, OrdenRepositorio>();
        services.AddScoped<IServicioOrdenes, ServicioOrdenes>();

        using var provider = services.BuildServiceProvider(new ServiceProviderOptions
        {
            ValidateScopes = true,
            ValidateOnBuild = true
        });

        using var scope = provider.CreateScope();
    }
}

```

```

var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();

var orden = new OrdenFabricacion
{
    NumeroOrden = "OF-M2-0001",
    Cliente = "Constructora del Norte",
    FechaCreacion = DateTime.UtcNow,
    FechaEntrega = DateTime.Today.AddDays(14),
    Estado = "Pendiente",
    Observaciones = "Orden de validación M2",
    IsDeleted = false,
    Planchas =
    {
        new PlanchaAcero
        {
            Espesor = 10.5,
            Ancho = 1500,
            Largo = 3000,
            Peso = 371.250m,
            Activa = true,
        }
    },
    Detalle = new DetalleOrden
    {
        ComposicionQuimica = "C: 0.45%, Mn: 0.75%",
        TemperaturaColada = 1550.5,
        Notas = "Colada principal"
    },
    Certificado = new CertificadoCalidad
    {
        NumeroCertificado = "CERT-0001",
        FechaEmision = DateTime.Today,
        OrganismoCertificador = "Laboratorio Aceria"
    },
};

var aleacion = new Aleacion
{
    Nombre = "AISI 1045",
   Codigo = "A1045",
    PorcentajeCarbono = 0.45,
    PorcentajeManganeso = 0.75,
    Descripcion = "Acero medio en carbono"
};
orden OrdenesAleaciones.Add(new OrdenAleacion
{
    Aleacion = aleacion,
    CantidadUtilizada = 1500.500m,
    EstadoRelacion = "Activa"
});
context OrdenesFabricacion.Add(orden);
context EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa",
Activo = true, IsDeleted = false });
context.SaveChanges();

var id = orden.Id;
orden.IsDeleted = true;
orden.DeletedAt = DateTime.UtcNow;
context.SaveChanges();
var visibles = context OrdenesFabricacion.Count();
var todas = context OrdenesFabricacion.IgnoreQueryFilters().Count();
var restaurable = context OrdenesFabricacion.IgnoreQueryFilters().Single(o => o.Id == id);
restaurable.IsDeleted = false;
restaurable.DeletedAt = null;
context.SaveChanges();

global::System.Console.WriteLine($"2.11 OK | Tras borrar visibles: {visibles} | Totales: {todas} |
Restaurada: {context OrdenesFabricacion.Any(o => o.Id == id)}");
}
}

```

Explicación línea a línea del código acumulativo 2.11

Las referencias usan la numeración real de `M02/PROYECTO/2.11/Program.cs`. Se explican todas las líneas no vacías del archivo que compila.

Línea 1: `using System.ComponentModel.DataAnnotations;` → importa el espacio de nombres necesario para este archivo.

Línea 2: `using System.ComponentModel.DataAnnotations.Schema;` → importa el espacio de nombres necesario para este archivo.

Línea 3: `using Microsoft.EntityFrameworkCore;` → importa el espacio de nombres necesario para este archivo.

Línea 4: `using Microsoft.Extensions.Configuration;` → importa el espacio de nombres necesario para este archivo.

Línea 5: `using Microsoft.Extensions.DependencyInjection;` → importa el espacio de nombres necesario para este archivo.

Línea 6: `using Microsoft.Extensions.Logging;` → importa el espacio de nombres necesario para este archivo.

Línea 8: `namespace AceriaData.ConsoleApp;` → declara el espacio de nombres del estado acumulativo.

Línea 10: `[Table("OrdenesFabricacion")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 11: `public class OrdenFabricacion` → declara la clase `OrdenFabricacion` que forma parte del estado acumulativo.

Línea 12: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 13: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 14: `public int Id { get; set; }` → declara la propiedad `Id` de tipo `int` usada por el modelo o la lógica de negocio.

Línea 15: `[Required]` → marca la propiedad siguiente como requerida.

Línea 16: `[MaxLength(50)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 17: `public string NumeroOrden { get; set; } = string.Empty;` → declara la propiedad `NumeroOrden` de tipo `string` usada por el modelo o la lógica de negocio.

Línea 18: `[Required]` → marca la propiedad siguiente como requerida.

Línea 19: `[MaxLength(200)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 20: `public string Cliente { get; set; } = string.Empty;` → declara la propiedad `Cliente` de tipo `string` usada por el modelo o la lógica de negocio.

Línea 21: `public DateTime FechaCreacion { get; set; }` → declara la propiedad `FechaCreacion` de tipo `DateTime` usada por el modelo o la lógica de negocio.

Línea 22: `public DateTime? FechaEntrega { get; set; }` → declara la propiedad `FechaEntrega` de tipo `DateTime?` usada por el modelo o la lógica de negocio.

Línea 23: `[Required]` → marca la propiedad siguiente como requerida.

Línea 24: `[MaxLength(50)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 25: `public string Estado { get; set; } = "Pendiente";` → declara la propiedad `Estado` de tipo `string` usada por el modelo o la lógica de negocio.

Línea 26: `[MaxLength(500)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 27: `public string? Observaciones { get; set; }` → declara la propiedad Observaciones de tipo `string?` usada por el modelo o la lógica de negocio.

Línea 28: `public List<PlanchaAcero> Planchas { get; set; } = new();` → declara la propiedad Planchas de tipo `List` usada por el modelo o la lógica de negocio.

Línea 29: `public DetalleOrden? Detalle { get; set; }` → declara la propiedad Detalle de tipo `DetalleOrden?` usada por el modelo o la lógica de negocio.

Línea 30: `public CertificadoCalidad? Certificado { get; set; }` → declara la propiedad Certificado de tipo `CertificadoCalidad?` usada por el modelo o la lógica de negocio.

Línea 31: `public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();` → declara la propiedad OrdenesAleaciones de tipo `List` usada por el modelo o la lógica de negocio.

Línea 32: `public bool IsDeleted { get; set; }` → declara la propiedad IsDeleted de tipo `bool` usada por el modelo o la lógica de negocio.

Línea 33: `public DateTime? DeletedAt { get; set; }` → declara la propiedad DeletedAt de tipo `DateTime?` usada por el modelo o la lógica de negocio.

Línea 34: `}` → cierra el bloque de código actual.

Línea 36: `[Table("PlanchasAcero")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 37: `public class PlanchaAcero` → declara la clase PlanchaAcero que forma parte del estado acumulativo.

Línea 38: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 39: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 40: `public int Id { get; set; }` → declara la propiedad Id de tipo `int` usada por el modelo o la lógica de negocio.

Línea 41: `public int OrdenId { get; set; }` → declara la propiedad OrdenId de tipo `int` usada por el modelo o la lógica de negocio.

Línea 42: `public double Espesor { get; set; }` → declara la propiedad Espesor de tipo `double` usada por el modelo o la lógica de negocio.

Línea 43: `public double Ancho { get; set; }` → declara la propiedad Ancho de tipo `double` usada por el modelo o la lógica de negocio.

Línea 44: `public double Largo { get; set; }` → declara la propiedad Largo de tipo `double` usada por el modelo o la lógica de negocio.

Línea 45: `[Precision(18, 3)]` → configura precisión y escala para la propiedad numérica siguiente.

Línea 46: `public decimal Peso { get; set; }` → declara la propiedad Peso de tipo `decimal` usada por el modelo o la lógica de negocio.

Línea 47: `public bool Activa { get; set; } = true;` → declara la propiedad Activa de tipo `bool` usada por el modelo o la lógica de negocio.

Línea 48: `[ForeignKey(nameof(OrdenId))]` → vincula la navegación siguiente con su clave foránea.

Línea 49: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad Orden de tipo `OrdenFabricacion` usada por el modelo o la lógica de negocio.

Línea 50: `public bool IsDeleted { get; set; }` → declara la propiedad IsDeleted de tipo `bool` usada por el modelo o la lógica de negocio.

Línea 51: `public DateTime? DeletedAt { get; set; }` → declara la propiedad DeletedAt de tipo `DateTime?` usada por el modelo o la lógica de negocio.

Línea 52: `}` → cierra el bloque de código actual.

Línea 54: `[Table("Aleaciones")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 55: `public class Aleacion` → declara la clase Aleacion que forma parte del estado acumulativo.

Línea 56: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 57: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 58: `public int Id { get; set; }` → declara la propiedad Id de tipo int usada por el modelo o la lógica de negocio.

Línea 59: `[Required]` → marca la propiedad siguiente como requerida.

Línea 60: `[MaxLength(100)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 61: `public string Nombre { get; set; } = string.Empty;` → declara la propiedad Nombre de tipo string usada por el modelo o la lógica de negocio.

Línea 62: `[Required]` → marca la propiedad siguiente como requerida.

Línea 63: `[MaxLength(20)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 64: `public string Codigo { get; set; } = string.Empty;` → declara la propiedad Codigo de tipo string usada por el modelo o la lógica de negocio.

Línea 65: `public double PorcentajeCarbono { get; set; }` → declara la propiedad PorcentajeCarbono de tipo double usada por el modelo o la lógica de negocio.

Línea 66: `public double PorcentajeManganeso { get; set; }` → declara la propiedad PorcentajeManganeso de tipo double usada por el modelo o la lógica de negocio.

Línea 67: `[MaxLength(500)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 68: `public string? Descripcion { get; set; }` → declara la propiedad Descripcion de tipo string? usada por el modelo o la lógica de negocio.

Línea 69: `public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();` → declara la propiedad OrdenesAleaciones de tipo List usada por el modelo o la lógica de negocio.

Línea 70: `public bool IsDeleted { get; set; }` → declara la propiedad IsDeleted de tipo bool usada por el modelo o la lógica de negocio.

Línea 71: `public DateTime? DeletedAt { get; set; }` → declara la propiedad DeletedAt de tipo DateTime? usada por el modelo o la lógica de negocio.

Línea 72: `}` → cierra el bloque de código actual.

Línea 74: `[Table("EstadosOrden")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 75: `public class EstadoOrden` → declara la clase EstadoOrden que forma parte del estado acumulativo.

Línea 76: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 77: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 78: `public int Id { get; set; }` → declara la propiedad Id de tipo int usada por el modelo o la lógica de negocio.

Línea 79: `[Required]` → marca la propiedad siguiente como requerida.

Línea 80: `[MaxLength(50)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 81: `public string Nombre { get; set; } = string.Empty;` → declara la propiedad Nombre de tipo string usada por el modelo o la lógica de negocio.

Línea 82: `[MaxLength(250)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 83: `public string Descripcion { get; set; } = string.Empty;` → declara la propiedad Descripcion de tipo string usada por el modelo o la lógica de negocio.

Línea 84: `public bool Activo { get; set; } = true;` → declara la propiedad Activo de tipo bool usada por el modelo o la lógica de negocio.

Línea 85: `public bool IsDeleted { get; set; }` → declara la propiedad IsDeleted de tipo bool usada por el modelo o la lógica de negocio.

Línea 86: `public DateTime? DeletedAt { get; set; }` → declara la propiedad DeletedAt de tipo DateTime? usada por el modelo o la lógica de negocio.

Línea 87: `}` → cierra el bloque de código actual.

Línea 89: `[Table("DetallesOrden")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 90: `public class DetalleOrden` → declara la clase DetalleOrden que forma parte del estado acumulativo.

Línea 91: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 92: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 93: `public int Id { get; set; }` → declara la propiedad Id de tipo int usada por el modelo o la lógica de negocio.

Línea 94: `public int OrdenId { get; set; }` → declara la propiedad OrdenId de tipo int usada por el modelo o la lógica de negocio.

Línea 95: `[Required]` → marca la propiedad siguiente como requerida.

Línea 96: `[MaxLength(200)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 97: `public string ComposicionQuimica { get; set; } = string.Empty;` → declara la propiedad ComposicionQuimica de tipo string usada por el modelo o la lógica de negocio.

Línea 98: `public double TemperaturaColada { get; set; }` → declara la propiedad TemperaturaColada de tipo double usada por el modelo o la lógica de negocio.

Línea 99: `[MaxLength(500)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 100: `public string? Notas { get; set; }` → declara la propiedad Notas de tipo string? usada por el modelo o la lógica de negocio.

Línea 101: `[ForeignKey(nameof(OrdenId))]` → vincula la navegación siguiente con su clave foránea.

Línea 102: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad Orden de tipo OrdenFabricacion usada por el modelo o la lógica de negocio.

Línea 103: `}` → cierra el bloque de código actual.

Línea 105: `[Table("CertificadosCalidad")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 106: `public class CertificadoCalidad` → declara la clase CertificadoCalidad que forma parte del estado acumulativo.

Línea 107: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 108: `[Key]` → marca la propiedad siguiente como clave primaria simple.

Línea 109: `public int Id { get; set; }` → declara la propiedad Id de tipo int usada por el modelo o la lógica de negocio.

Línea 110: `public int OrdenId { get; set; }` → declara la propiedad `OrdenId` de tipo `int` usada por el modelo o la lógica de negocio.

Línea 111: `[Required]` → marca la propiedad siguiente como requerida.

Línea 112: `[MaxLength(50)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 113: `public string NumeroCertificado { get; set; } = string.Empty;` → declara la propiedad `NumeroCertificado` de tipo `string` usada por el modelo o la lógica de negocio.

Línea 114: `public DateTime FechaEmision { get; set; }` → declara la propiedad `FechaEmision` de tipo `DateTime` usada por el modelo o la lógica de negocio.

Línea 115: `[Required]` → marca la propiedad siguiente como requerida.

Línea 116: `[MaxLength(100)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 117: `public string OrganismoCertificador { get; set; } = string.Empty;` → declara la propiedad `OrganismoCertificador` de tipo `string` usada por el modelo o la lógica de negocio.

Línea 118: `[ForeignKey(nameof(OrdenId))]` → vincula la navegación siguiente con su clave foránea.

Línea 119: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad `Orden` de tipo `OrdenFabricacion` usada por el modelo o la lógica de negocio.

Línea 120: `}` → cierra el bloque de código actual.

Línea 122: `[PrimaryKey(nameof(OrdenFabricacionId), nameof(AleacionId))]` → declara una clave primaria compuesta.

Línea 123: `[Table("OrdenesAleaciones")]` → fija mediante Data Annotations la tabla relacional de la entidad siguiente.

Línea 124: `public class OrdenAleacion` → declara la clase `OrdenAleacion` que forma parte del estado acumulativo.

Línea 125: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 126: `public int OrdenFabricacionId { get; set; }` → declara la propiedad `OrdenFabricacionId` de tipo `int` usada por el modelo o la lógica de negocio.

Línea 127: `public int AleacionId { get; set; }` → declara la propiedad `AleacionId` de tipo `int` usada por el modelo o la lógica de negocio.

Línea 128: `public DateTime FechaAsignacion { get; set; } = DateTime.Now;` → declara la propiedad `FechaAsignacion` de tipo `DateTime` usada por el modelo o la lógica de negocio.

Línea 129: `[Precision(18, 3)]` → configura precisión y escala para la propiedad numérica siguiente.

Línea 130: `public decimal CantidadUtilizada { get; set; }` → declara la propiedad `CantidadUtilizada` de tipo `decimal` usada por el modelo o la lógica de negocio.

Línea 131: `[MaxLength(20)]` → limita la longitud máxima persistida de la propiedad siguiente.

Línea 132: `public string EstadoRelacion { get; set; } = "Activa";` → declara la propiedad `EstadoRelacion` de tipo `string` usada por el modelo o la lógica de negocio.

Línea 133: `public OrdenFabricacion Orden { get; set; } = null!;` → declara la propiedad `Orden` de tipo `OrdenFabricacion` usada por el modelo o la lógica de negocio.

Línea 134: `public Aleacion Aleacion { get; set; } = null!;` → declara la propiedad `Aleacion` de tipo `Aleacion` usada por el modelo o la lógica de negocio.

Línea 135: `}` → cierra el bloque de código actual.

Línea 137: `public class AceriaDbContext : DbContext` → declara la clase `AceriaDbContext` que forma parte del estado acumulativo.

Línea 138: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 139:

`public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }` → recibe las opciones del DbContext y las pasa a la clase base.

Línea 141: `public DbSet<OrdenFabricacion> OrdenesFabricacion => Set<OrdenFabricacion>();` → expone un conjunto de entidades para consultas y persistencia.

Línea 142: `public DbSet<PlanchaAcero> PlanchasAcero => Set<PlanchaAcero>();` → expone un conjunto de entidades para consultas y persistencia.

Línea 143: `public DbSet<Aleacion> Aleaciones => Set<Aleacion>();` → expone un conjunto de entidades para consultas y persistencia.

Línea 144: `public DbSet<EstadoOrden> EstadosOrden => Set<EstadoOrden>();` → expone un conjunto de entidades para consultas y persistencia.

Línea 145: `public DbSet<DetalleOrden> DetallesOrden => Set<DetalleOrden>();` → expone un conjunto de entidades para consultas y persistencia.

Línea 146: `public DbSet<CertificadoCalidad> CertificadosCalidad => Set<CertificadoCalidad>();` → expone un conjunto de entidades para consultas y persistencia.

Línea 147: `public DbSet<OrdenAleacion> OrdenesAleaciones => Set<OrdenAleacion>();` → expone un conjunto de entidades para consultas y persistencia.

Línea 150: `protected override void OnModelCreating(ModelBuilder modelBuilder)` → entra en la configuración explícita del modelo.

Línea 151: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 152: `base.OnModelCreating(modelBuilder);` → entra en la configuración explícita del modelo.

Línea 153: `modelBuilder.Entity<OrdenFabricacion>(entity =>` → selecciona una entidad para configurarla con Fluent API.

Línea 154: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 155: `entity.ToTable("OrdenesFabricacion");` → asigna explícitamente la tabla usada por la entidad.

Línea 156: `entity.HasKey(o => o.Id);` → define la clave primaria.

Línea 157: `entity.Property(o => o.NumeroOrden).IsRequired().HasMaxLength(50);` → selecciona una propiedad escalar para continuar su configuración.

Línea 158: `entity.Property(o => o.Cliente).IsRequired().HasMaxLength(200);` → selecciona una propiedad escalar para continuar su configuración.

Línea 159: `entity.Property(o => o.FechaCreacion).HasDefaultValueSql("GETDATE()");` → selecciona una propiedad escalar para continuar su configuración.

Línea 160: `entity.Property(o => o.Estado).IsRequired().HasMaxLength(50).HasDefaultValue("Pendiente");` → selecciona una propiedad escalar para continuar su configuración.

Línea 161: `entity.Property(o => o.Observaciones).HasMaxLength(500);` → selecciona una propiedad escalar para continuar su configuración.

Línea 162: `entity.HasAlternateKey(o => o.NumeroOrden).HasName("AK_OrdenesFabricacion_NumeroOrden");` → define una clave alternativa única.

Línea 163: `entity.HasIndex(o => o.Cliente).HasDatabaseName("IX_OrdenesFabricacion_Cliente");` → define un índice sobre las propiedades indicadas.

Línea 164: `entity.HasIndex(o => new { o.Cliente, o.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Cliente_FechaCreacion");` → define un índice sobre las propiedades indicadas.

Línea 165: `entity.HasIndex(o => o.FechaEntrega).HasFilter("[Estado] = 'Pendiente')").HasDatabaseName("IX_OrdenesFabricacion_FechaEntrega_Pendientes");` → define un índice sobre las propiedades indicadas.

Línea 166: `entity.HasIndex(o => o.Estado).IncludeProperties(o => new { o.NumeroOrden, o.Cliente, o.FechaCreacion }).HasDatabaseName("IX_OrdenesFabricacion_Estado_Incluye");` → define un índice sobre las propiedades indicadas.

Línea 167: `entity.HasQueryFilter(o => !o.IsDeleted && o.Estado != "Cancelada");` → aplica un filtro global de consulta.

Línea 168: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 169: `modelBuilder.Entity<PlanchaAcero>(entity =>` → selecciona una entidad para configurarla con Fluent API.

Línea 170: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 171: `entity.ToTable("PlanchasAcero", t =>` → asigna explícitamente la tabla usada por la entidad.

Línea 172: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 173: `t.HasCheckConstraint("CK_PlanchasAcero_Espesor", "[Espesor] > 0");` → añade una restricción CHECK en SQL Server.

Línea 174: `t.HasCheckConstraint("CK_PlanchasAcero_Ancho", "[Ancho] > 0");` → añade una restricción CHECK en SQL Server.

Línea 175: `t.HasCheckConstraint("CK_PlanchasAcero_Largo", "[Largo] > 0");` → añade una restricción CHECK en SQL Server.

Línea 176: `t.HasCheckConstraint("CK_PlanchasAcero_Peso", "[Peso] > 0");` → añade una restricción CHECK en SQL Server.

Línea 177: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 178: `entity.HasKey(x => x.Id);` → define la clave primaria.

Línea 179: `entity.Property(x => x.Peso).HasPrecision(18, 3);` → selecciona una propiedad escalar para continuar su configuración.

Línea 180: `entity.Property(x => x.Activa).HasDefaultValue(true);` → selecciona una propiedad escalar para continuar su configuración.

Línea 181: `entity.HasOne(x => x.Orden)` → inicia la configuración de una navegación de referencia.

Línea 182: `.WithMany(o => o.Planchas)` → establece el extremo de colección de una relación uno-a-muchos.

Línea 183: `.HasForeignKey(x => x.OrdenId)` → declara explícitamente la clave foránea.

Línea 184: `.OnDelete(DeleteBehavior.Cascade)` → define el comportamiento de eliminación de la relación.

Línea 185: `.IsRequired();` → marca la propiedad o relación como obligatoria.

Línea 186: `entity.HasIndex(x => new { x.OrdenId, x.Activa }).HasDatabaseName("IX_PlanchasAcero_OrdenId_Activa");` → define un índice sobre las propiedades indicadas.

Línea 187: `entity.HasIndex(x => x.Espesor).HasFilter("[Activa] = 1").HasDatabaseName("IX_PlanchasAcero_Espesor_Activas");` → define un índice sobre las propiedades indicadas.

Línea 188: `entity.HasQueryFilter(x => !x.IsDeleted && x.Activa);` → aplica un filtro global de consulta.

Línea 189: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 190: `modelBuilder.Entity<Aleacion>(entity =>` → selecciona una entidad para configurarla con Fluent API.

Línea 191: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 192: `entity.ToTable("Aleaciones", t =>` → asigna explícitamente la tabla usada por la entidad.

Línea 193: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 194: `t.HasCheckConstraint("CK_Aleaciones_PorcentajeCarbono", "[PorcentajeCarbono] >= 0 AND [PorcentajeCarbono] <= 2");` → añade una restricción CHECK en SQL Server.

Línea 195: `t.HasCheckConstraint("CK_Aleaciones_PorcentajeManganeso", "[PorcentajeManganeso] >= 0 AND [PorcentajeManganeso] <= 5");` → añade una restricción CHECK en SQL Server.

Línea 196: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 197: `entity.HasKey(a => a.Id);` → define la clave primaria.

Línea 198: `entity.Property(a => a.Nombre).IsRequired().HasMaxLength(100);` → selecciona una propiedad escalar para continuar su configuración.

Línea 199: `entity.Property(a => a.Codigo).IsRequired().HasMaxLength(20);` → selecciona una propiedad escalar para continuar su configuración.

Línea 200: `entity.Property(a => a.Descripcion).HasMaxLength(500);` → selecciona una propiedad escalar para continuar su configuración.

Línea 201: `entity.HasAlternateKey(a => a.Codigo).HasName("AK_Aleaciones_Codigo");` → define una clave alternativa única.

Línea 202: `entity.HasIndex(a => a.Nombre).HasDatabaseName("IX_Aleaciones_Nombre");` → define un índice sobre las propiedades indicadas.

Línea 203: `entity.HasIndex(a => new { a.PorcentajeCarbono, a.PorcentajeManganeso }).HasDatabaseName("IX_Aleaciones_Porcentajes");` → define un índice sobre las propiedades indicadas.

Línea 204: `entity.HasQueryFilter(a => !a.IsDeleted);` → aplica un filtro global de consulta.

Línea 205: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 206: `modelBuilder.Entity<EstadoOrden>(entity =>` → selecciona una entidad para configurarla con Fluent API.

Línea 207: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 208: `entity.ToTable("EstadosOrden");` → asigna explícitamente la tabla usada por la entidad.

Línea 209: `entity.HasKey(e => e.Id);` → define la clave primaria.

Línea 210: `entity.Property(e => e.Nombre).IsRequired().HasMaxLength(50);` → selecciona una propiedad escalar para continuar su configuración.

Línea 211: `entity.Property(e => e.Descripcion).HasMaxLength(250);` → selecciona una propiedad escalar para continuar su configuración.

Línea 212: `entity.Property(e => e.Activo).HasDefaultValue(true);` → selecciona una propiedad escalar para continuar su configuración.

Línea 213: `entity.HasQueryFilter(e => !e.IsDeleted && e.Activo);` → aplica un filtro global de consulta.

Línea 214: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 215: `modelBuilder.Entity<DetalleOrden>(entity =>` → selecciona una entidad para configurarla con Fluent API.

Línea 216: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 217: `entity.ToTable("DetallesOrden");` → asigna explícitamente la tabla usada por la entidad.

Línea 218: `entity.HasKey(d => d.Id);` → define la clave primaria.

Línea 219: `entity.Property(d => d.ComposicionQuimica).IsRequired().HasMaxLength(200);` → selecciona una propiedad escalar para continuar su configuración.

Línea 220: `entity.Property(d => d.Notas).HasMaxLength(500);` → selecciona una propiedad escalar para continuar su configuración.

Línea 221: `entity.HasOne(d => d.Orden)` → inicia la configuración de una navegación de referencia.

Línea 222: `.WithOne(o => o.Detalle)` → establece el extremo de referencia de una relación uno-a-uno.

Línea 223: `.HasForeignKey<DetalleOrden>(d => d.OrdenId)` → declara explícitamente la clave foránea.

Línea 224: `.OnDelete(DeleteBehavior.Cascade)` → define el comportamiento de eliminación de la relación.

Línea 225: `.IsRequired();` → marca la propiedad o relación como obligatoria.

Línea 226: `entity.HasQueryFilter(d => !d.Orden.IsDeleted && d.Orden.Estado != "Cancelada");` → aplica un filtro global de consulta.

Línea 227: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 229: `modelBuilder.Entity<CertificadoCalidad>(entity =>` → selecciona una entidad para configurarla con Fluent API.

Línea 230: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 231: `entity.ToTable("CertificadosCalidad");` → asigna explícitamente la tabla usada por la entidad.

Línea 232: `entity.HasKey(c => c.Id);` → define la clave primaria.

Línea 233: `entity.Property(c => c.NumeroCertificado).IsRequired().HasMaxLength(50);` → selecciona una propiedad escalar para continuar su configuración.

Línea 234: `entity.Property(c => c.OrganismoCertificador).IsRequired().HasMaxLength(100);` → selecciona una propiedad escalar para continuar su configuración.

Línea 235: `entity.HasAlternateKey(c => c.NumeroCertificado).HasName("AK_CertificadosCalidad_NumeroCertificado");` → define una clave alternativa única.

Línea 236: `entity.HasIndex(c => c.FechaEmision).HasDatabaseName("IX_CertificadosCalidad_FechaEmision");` → define un índice sobre las propiedades indicadas.

Línea 237: `entity.HasOne(c => c.Orden)` → inicia la configuración de una navegación de referencia.

Línea 238: `.WithOne(o => o.Certificado)` → establece el extremo de referencia de una relación uno-a-uno.

Línea 239: `.HasForeignKey<CertificadoCalidad>(c => c.OrdenId)` → declara explícitamente la clave foránea.

Línea 240: `.OnDelete(DeleteBehavior.Cascade)` → define el comportamiento de eliminación de la relación.

Línea 241: `.IsRequired();` → marca la propiedad o relación como obligatoria.

Línea 242: `entity.HasQueryFilter(c => !c.Orden.IsDeleted && c.Orden.Estado != "Cancelada");` → aplica un filtro global de consulta.

Línea 243: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 244: `modelBuilder.Entity<OrdenAleacion>(entity =>` → selecciona una entidad para configurarla con Fluent API.

Línea 245: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 246: `entity.ToTable("OrdenesAleaciones");` → asigna explícitamente la tabla usada por la entidad.

Línea 247: `entity.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });` → define la clave primaria.

Línea 248: `entity.Property(x => x.FechaAsignacion).HasDefaultValueSql("GETDATE()");` → selecciona una propiedad escalar para continuar su configuración.

Línea 249: `entity.Property(x => x.CantidadUtilizada).HasPrecision(18, 3);` → selecciona una propiedad escalar para continuar su configuración.

Línea 250: `entity.Property(x => x.EstadoRelacion).IsRequired().HasMaxLength(20).HasDefaultValue("Activa");` → selecciona una propiedad escalar para continuar su configuración.

Línea 251: `entity.HasOne(x => x.Orden)` → inicia la configuración de una navegación de referencia.

Línea 252: `.WithMany(o => o.OrdenesAleaciones)` → establece el extremo de colección de una relación uno-a-muchos.

Línea 253: `.HasForeignKey(x => x.OrdenFabricacionId)` → declara explícitamente la clave foránea.

Línea 254: `.OnDelete(DeleteBehavior.Cascade);` → define el comportamiento de eliminación de la relación.

Línea 255: `entity.HasOne(x => x.Aleacion)` → inicia la configuración de una navegación de referencia.

Línea 256: `.WithMany(a => a.OrdenesAleaciones)` → establece el extremo de colección de una relación uno-a-muchos.

Línea 257: `.HasForeignKey(x => x.AleacionId)` → declara explícitamente la clave foránea.

Línea 258: `.OnDelete(DeleteBehavior.Restrict);` → define el comportamiento de eliminación de la relación.

Línea 259: `entity.HasIndex(x => x.AleacionId).HasDatabaseName("IX_OrdenesAleaciones_AleacionId");` → define un índice sobre las propiedades indicadas.

Línea 260: `entity.HasIndex(x => x.EstadoRelacion).HasFilter("[EstadoRelacion] = 'Activa'").HasDatabaseName("IX_OrdenesAleaciones_EstadoRelacion_Activas");` → define un índice sobre las propiedades indicadas.

Línea 261: `entity.HasQueryFilter(x => x.EstadoRelacion == "Activa" && !x.Orden.IsDeleted && !x.Aleacion.IsDeleted);` → aplica un filtro global de consulta.

Línea 262: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 263: `}` → cierra el bloque de código actual.

Línea 264: `}` → cierra el bloque de código actual.

Línea 266: `public interface IOrdenRepositorio` → declara el contrato `IOrdenRepositorio` usado por la aplicación.

Línea 267: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 268: `List<OrdenFabricacion> ObtenerTodas();` → declara en el contrato la operación que debe devolver todas las órdenes visibles para el repositorio.

Línea 269: `OrdenFabricacion? ObtenerPorId(int id);` → declara en el contrato la búsqueda de una orden por su clave primaria y permite devolver null si no existe.

Línea 270: `OrdenFabricacion? ObtenerPorNumero(string numeroOrden);` → declara en el contrato la búsqueda por el identificador de negocio `NumeroOrden`.

Línea 271: `void Agregar(OrdenFabricacion orden);` → declara la operación que incorpora una orden al repositorio para que EF Core la siga como nueva.

Línea 272: `void Eliminar(OrdenFabricacion orden);` → declara la operación que marca una orden para eliminación física cuando el escenario lo requiere.

Línea 273: `void EliminarLogicamente(int id);` → declara en el contrato del repositorio la operación de Soft Delete por identificador.

Línea 274: `void Restaurar(int id);` → declara en el contrato del repositorio la operación que revierte un Soft Delete.

Línea 276: `int Guardar();` → declara la operación que confirma los cambios pendientes y devuelve el número de entradas afectadas.

Línea 277: `}` → cierra el bloque de código actual.

Línea 279: `public sealed class OrdenRepositorio : IOrdenRepositorio` → declara la clase `OrdenRepositorio` que forma parte del estado acumulativo.

Línea 280: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 281: `private readonly AceriaDbContext _context;` → declara el campo privado `_context` de tipo `AceriaDbContext`.

Línea 282: `public OrdenRepositorio(AceriaDbContext context) => _context = context;` → inyecta `AceriaDbContext` en el repositorio y lo conserva como dependencia de persistencia.

Línea 284: `public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.OrderBy(o => o.Id).ToList();` → ordena el resultado antes de materializarlo.

Línea 285: `public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);` → implementa la búsqueda por clave primaria mediante `Find`, aprovechando primero el `ChangeTracker` y después la base de datos.

Línea 286: `public OrdenFabricacion? ObtenerPorNumero(string numeroOrden) => _context.OrdenesFabricacion.FirstOrDefault(o => o.NumeroOrden == numeroOrden);` → recupera la primera coincidencia o null.

Línea 287: `public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);` → registra una entidad nueva para insertarla al guardar.

Línea 288: `public void Eliminar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Remove(orden);` → marca una entidad para borrado al guardar.

Línea 289: `public void EliminarLogicamente(int id)` → implementa la operación que localiza una orden y cambia sus campos de borrado lógico.

Línea 290: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 291: `var orden = _context.OrdenesFabricacion.FirstOrDefault(o => o.Id == id);` → recupera la primera coincidencia o null.

Línea 292: `if (orden is null) return;` → evalúa una condición antes de ejecutar el bloque asociado.

Línea 293: `orden.IsDeleted = true;` → marca la orden como eliminada lógicamente sin borrar su fila física.

Línea 294: `orden.DeletedAt = DateTime.UtcNow;` → registra en UTC el instante en que se realizó el borrado lógico.

Línea 295: `}` → cierra el bloque de código actual.

Línea 297: `public void Restaurar(int id)` → implementa la restauración cargando también entidades ocultas por el filtro global.

Línea 298: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 299: `var orden = _context.OrdenesFabricacion.IgnoreQueryFilters().FirstOrDefault(o => o.Id == id);` → omite de forma explícita los filtros globales para esta consulta.

Línea 300: `if (orden is null) return;` → evalúa una condición antes de ejecutar el bloque asociado.

Línea 301: `orden.IsDeleted = false;` → desactiva la marca de borrado lógico para volver a hacer visible la orden.

Línea 302: `orden.DeletedAt = null;` → elimina la fecha de borrado porque la orden vuelve al estado activo.

Línea 303: `}` → cierra el bloque de código actual.

Línea 305: `public int Guardar() => _context.SaveChanges();` → confirma en SQL Server los cambios del ChangeTracker.

Línea 306: `}` → cierra el bloque de código actual.

Línea 308: `public interface IServicioOrdenes` → declara el contrato IServicioOrdenes usado por la aplicación.

Línea 309: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 310: `string ObtenerResumen(int ordenId);` → declara el servicio que construye un resumen de una orden a partir de su identificador.

Línea 311: `}` → cierra el bloque de código actual.

Línea 313: `public sealed class ServicioOrdenes : IServicioOrdenes` → declara la clase ServicioOrdenes que forma parte del estado acumulativo.

Línea 314: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 315: `private readonly IOrdenRepositorio _repositorio;` → declara el campo privado _repositorio de tipo IOrdenRepositorio.

Línea 316: `private readonly AceriaDbContext _context;` → declara el campo privado _context de tipo AceriaDbContext.

Línea 317: `public ServicioOrdenes(IOrdenRepositorio repositorio, AceriaDbContext context)` → declara el constructor que recibe por inyección el repositorio de órdenes y el DbContext.

Línea 318: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 319: `_repositorio = repositorio;` → guarda en el servicio el repositorio inyectado para reutilizarlo en sus operaciones.

Línea 320: `_context = context;` → guarda el DbContext inyectado para consultar los datos necesarios durante el servicio.

Línea 321: `}` → cierra el bloque de código actual.

Línea 323: `public string ObtenerResumen(int ordenId)` → declara el método que obtiene la orden y compone el resumen solicitado por el servicio.

Línea 324: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 325: `var orden = _repositorio.ObtenerPorId(ordenId);` → declara una variable local y almacena el resultado de la expresión.

Línea 326: `if (orden is null) return $"Orden {ordenId} no encontrada";` → evalúa una condición antes de ejecutar el bloque asociado.

Línea 327: `var totalPlanchas = _context.PlanchasAcero.Count(x => x.OrdenId == ordenId);` → cuenta las filas que cumplen la consulta.

Línea 328: `return $"Orden {orden.NumeroOrden} | Cliente: {orden.Cliente} | Planchas: {totalPlanchas}";` → devuelve el resultado al código llamador.

Línea 329: `}` → cierra el bloque de código actual.

Línea 330: `}` → cierra el bloque de código actual.

Línea 332: `public static class Program` → declara la clase Program que forma parte del estado acumulativo.

Línea 333: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 334: `public static void Main()` → declara el punto de entrada ejecutable de la aplicación de consola.

Línea 335: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 336: `var configuration = new ConfigurationBuilder()` → inicia la construcción de la configuración de la aplicación.

Línea 337: `.SetBasePath(AppContext.BaseDirectory)` → establece la ruta base para los archivos de configuración.

Línea 338: `.AddJsonFile("appsettings.json", optional: false, reloadOnChange: false)` → incorpora appsettings.json.

Línea 339: `.AddEnvironmentVariables()` → incorpora variables de entorno.

Línea 340: `.Build();` → materializa el objeto construido por la cadena anterior.

Línea 342: `var connectionString = configuration.GetConnectionString("AceriaDB")` → obtiene la cadena AceriaDB desde configuración.

Línea 343: `?? throw new InvalidOperationException("No se encontró la cadena de conexión 'AceriaDB'.");` → interrumpe el arranque con un error explícito si la configuración no contiene ConnectionStrings:AceriaDB.

Línea 345: `var services = new ServiceCollection();` → crea la colección de servicios.

Línea 346: `services.AddDbContext<AceriaDbContext>(options =>` → registra el DbContext en inyección de dependencias.

Línea 347: `options` → continúa la configuración del DbContext sobre el parámetro options recibido por la lambda de AddDbContext.

Línea 348: `.UseSqlServer(connectionString, sql =>` → selecciona SQL Server como proveedor de EF Core.

Línea 349: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 350: `sql.EnableRetryOnFailure(5, TimeSpan.FromSeconds(10), null);` → habilita reintentos ante fallos transitorios.

Línea 351: `sql.CommandTimeout(60);` → establece el timeout de los comandos SQL.

Línea 352: `})` → cierra la lambda de configuración actual y devuelve el control a la llamada encadenada que la contiene.

Línea 353: `.LogTo(global::System.Console.WriteLine, new[] { "Microsoft.EntityFrameworkCore.Database.Command" }, LogLevel.Information)` → configura el logging de EF Core.

Línea 354: `.EnableDetailedErrors();` → activa errores detallados.

Línea 356: `services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();` → registra un servicio con ciclo de vida scoped.

Línea 357: `services.AddScoped<IServicioOrdenes, ServicioOrdenes>();` → registra un servicio con ciclo de vida scoped.

Línea 359: `using var provider = services.BuildServiceProvider(new ServiceProviderOptions` → construye el ServiceProvider con validación de servicios y garantiza su liberación al salir del ámbito mediante using var.

Línea 360: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 361: `ValidateScopes = true,` → activa la validación de ámbitos para detectar dependencias scoped usadas desde ámbitos incorrectos.

Línea 362: `ValidateOnBuild = true` → obliga al contenedor a validar las resoluciones registradas al construir el ServiceProvider.

Línea 363: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 365: `using var scope = provider.CreateScope();` → crea un ámbito de DI para resolver servicios scoped y lo libera automáticamente al terminar el bloque.

Línea 366: `var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();` → resuelve un servicio obligatorio del contenedor.

Línea 367: `context.Database.EnsureDeleted();` → reinicia la base del laboratorio para una ejecución reproducible.

Línea 368: `context.Database.Migrate();` → aplica el historial de migraciones pendiente.

Línea 370: `var orden = new OrdenFabricacion` → declara una variable local y almacena el resultado de la expresión.

Línea 371: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 372: `NumeroOrden = "OF-M2-0001",` → asigna el identificador de negocio usado por la orden de validación acumulativa del módulo.

Línea 373: `Cliente = "Constructora del Norte",` → asigna el cliente de la orden utilizada en el escenario práctico del punto.

Línea 374: `FechaCreacion = DateTime.UtcNow,` → asigna a la orden la fecha de creación en UTC para el escenario reproducible del punto.

Línea 375: `FechaEntrega = DateTime.Today.AddDays(14),` → establece una fecha de entrega prevista catorce días después de la fecha local actual.

Línea 376: `Estado = "Pendiente",` → inicializa la orden con el estado Pendiente, coherente con el flujo del laboratorio.

Línea 377: `Observaciones = "Orden de validación M2",` → añade texto opcional para comprobar el mapeo de Observaciones.

Línea 378: `IsDeleted = false,` → inicializa explícitamente la orden del escenario como no eliminada.

Línea 379: `Planchas =` → inicia el inicializador de la colección de planchas asociadas a la orden.

Línea 380: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 381: `new PlanchaAcero` → crea una nueva plancha dentro del inicializador de la colección Planchas de la orden.

Línea 382: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 383: `Espesor = 10.5,` → asigna a la plancha un espesor positivo para satisfacer las reglas del modelo.

Línea 384: `Ancho = 1500,` → asigna el ancho de la plancha usada en el escenario de persistencia.

Línea 385: `Largo = 3000,` → asigna el largo de la plancha usada en el escenario de persistencia.

Línea 386: `Peso = 371.250m,` → asigna un peso decimal con tres posiciones para validar la precisión configurada.

Línea 387: `Activa = true,` → marca la plancha como activa para que sea visible en los estados que aplican ese criterio.

Línea 388: `}` → cierra el bloque de código actual.

Línea 389: `},` → continúa el inicializador o la llamada del bloque actual.

Línea 390: `Detalle = new DetalleOrden` → crea el detalle uno-a-uno que se persistirá junto con la orden principal.

Línea 391: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 392: `ComposicionQuimica = "C: 0.45%, Mn: 0.75%",` → asigna la composición química utilizada para verificar la entidad DetalleOrden.

Línea 393: `TemperaturaColada = 1550.5,` → asigna la temperatura de colada del detalle de fabricación.

Línea 394: `Notas = "Colada principal"` → asigna las notas opcionales del detalle asociado a la orden.

Línea 395: `},` → continúa el inicializador o la llamada del bloque actual.

Línea 396: `Certificado = new CertificadoCalidad` → crea el certificado uno-a-uno asociado a la orden del escenario.

Línea 397: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 398: `NumeroCertificado = "CERT-0001",` → asigna el identificador natural del certificado de calidad.

Línea 399: `FechaEmision = DateTime.Today,` → asigna la fecha de emisión del certificado usado en la prueba.

Línea 400: `OrganismoCertificador = "Laboratorio Aceria"` → asigna el organismo que emite el certificado de calidad.

Línea 401: `},` → continúa el inicializador o la llamada del bloque actual.

Línea 402: `};` → cierra el inicializador del objeto y finaliza la instrucción.

Línea 404: `var aleacion = new Aleacion` → declara una variable local y almacena el resultado de la expresión.

Línea 405: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 406: `Nombre = "AISI 1045",` → asigna el nombre comercial de la aleación utilizada en el escenario.

Línea 407: `Codigo = "A1045",` → asigna el código natural único de la aleación.

Línea 408: `PorcentajeCarbono = 0.45,` → asigna el porcentaje de carbono dentro del rango permitido por la restricción del modelo.

Línea 409: `PorcentajeManganeso = 0.75`, → asigna el porcentaje de manganeso dentro del rango permitido por la restricción del modelo.

Línea 410: `Descripcion = "Acero medio en carbono"` → asigna una descripción opcional a la aleación.

Línea 411: `};` → cierra el inicializador del objeto y finaliza la instrucción.

Línea 412: `orden.OrdenesAleaciones.Add(new OrdenAleacion` → registra una entidad nueva para insertarla al guardar.

Línea 413: `{` → abre el bloque de código asociado a la declaración o instrucción anterior.

Línea 414: `Aleacion = aleacion`, → conecta la entidad intermedia `OrdenAleacion` con la aleación creada para el escenario.

Línea 415: `CantidadUtilizada = 1500.500m`, → registra la cantidad utilizada con tres decimales, coherente con `HasPrecision(18, 3)`.

Línea 416: `EstadoRelacion = "Activa"` → marca como activa la relación entre la orden y la aleación.

Línea 417: `});` → cierra la lambda o configuración encadenada y finaliza la llamada.

Línea 418: `context.OrdenesFabricacion.Add(orden);` → registra una entidad nueva para insertarla al guardar.

Línea 419: `context.EstadosOrden.Add(new EstadoOrden { Nombre = "Pendiente", Descripcion = "Orden activa", Activo = true, IsDeleted = false });` → registra una entidad nueva para insertarla al guardar.

Línea 420: `context.SaveChanges();` → confirma en SQL Server los cambios del `ChangeTracker`.

Línea 422: `var id = orden.Id;` → declara una variable local y almacena el resultado de la expresión.

Línea 423: `orden.IsDeleted = true;` → marca la orden como eliminada lógicamente sin borrar su fila física.

Línea 424: `orden.DeletedAt = DateTime.UtcNow;` → registra en UTC el instante en que se realizó el borrado lógico.

Línea 425: `context.SaveChanges();` → confirma en SQL Server los cambios del `ChangeTracker`.

Línea 426: `var visibles = context.OrdenesFabricacion.Count();` → cuenta las filas que cumplen la consulta.

Línea 427: `var todas = context.OrdenesFabricacion.IgnoreQueryFilters().Count();` → omite de forma explícita los filtros globales para esta consulta.

Línea 428: `var restaurable = context.OrdenesFabricacion.IgnoreQueryFilters().Single(o => o.Id == id);` → omite de forma explícita los filtros globales para esta consulta.

Línea 429: `restaurable.IsDeleted = false;` → restaura la entidad cargada ignorando filtros al desactivar su marca `IsDeleted`.

Línea 430: `restaurable.DeletedAt = null;` → limpia la fecha de eliminación de la entidad restaurada.

Línea 431: `context.SaveChanges();` → confirma en SQL Server los cambios del `ChangeTracker`.

Línea 432: `global::System.Console.WriteLine($"2.11 OK | Tras borrar visibles: {visibles} | Totales: {todas} | Restaurada: {context.OrdenesFabricacion.Any(o => o.Id == id)}");` → escribe la evidencia de ejecución usada para validar este estado.

Línea 433: `}` → cierra el bloque de código actual.

Línea 434: `}` → cierra el bloque de código actual.